

Popular Machine Learning Methods: Idea, Practice and Math

Part 2, Chapter 2, Section 3:
Logistic Regression

Yuxiao Huang

Data Science, Columbian College of Arts & Sciences
George Washington University

Spring 2026

Reference

- This set of slides was largely built on the following 7 wonderful books and a wide range of fabulous papers:
 - HML Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)
 - PML Python Machine Learning (3rd Edition)
 - ESL The Elements of Statistical Learning (2nd Edition)
 - PRML Pattern Recognition and Machine Learning
 - NND Neural Network Design (2nd Edition)
 - LFD Learning From Data
 - RL Reinforcement Learning: An Introduction (2nd Edition)
- For most materials covered in the slides, we will specify their corresponding books and papers for further reference.

Code Example & Case Study

- See related code examples in github repository:
[/p2_c2_s3_logistic_regression/code_example](#)
- See related case studies of Kaggle Competition in github repository:
[/p2_c2_s3_logistic_regression/case_study](#)

Table of Contents

- 1 Learning Objectives
- 2 Motivating Example
- 3 Logistic Regression
- 4 Sigmoid Regression
- 5 Sigmoid Regression in Sklearn
- 6 Training Sigmoid Regression
- 7 Softmax Regression
- 8 Softmax Regression in Sklearn
- 9 Training Softmax Regression
- 10 Class Imbalance: Revisit
- 11 Metrics for Classification
- 12 Appendix
- 13 Bibliography

Learning Objectives: Expectation

- It is expected to understand
 - the idea of Classification
 - the idea of three kinds of classification:
 - Binary Classification
 - Multiclass Classification
 - Multilabel Classification
 - the good practice for using sklearn LogisticRegression
 - the idea of Bernoulli and Categorical distribution
 - the idea of Binomial and Multinomial distribution
 - the idea of Maximum Likelihood Estimation, Kullback–Leibler Divergence and their connection
 - the idea of Binomial Cross Entropy and Multinomial Cross Entropy
 - the idea and implementation of two forms of Logistic Regression:
 - Sigmoid Regression
 - Softmax Regression
 - the idea of why both sigmoid and softmax regression suffer from class imbalance
 - the idea and good practice of three kinds of metrics for classification:
 - Confusion Matrix
 - Accuracy
 - Precision, Recall and F1-Score

Learning Objectives: Recommendation

- It is recommended to understand
 - the math of Bernoulli and Categorical distribution
 - the math of Binomial and Multinomial distribution
 - the math of Maximum Likelihood Estimation, Kullback–Leibler Divergence and their connection
 - the math of Binomial Cross Entropy and Multinomial Cross Entropy
 - the math of two forms of Logistic Regression:
 - Sigmoid Regression
 - Softmax Regression
 - the math of why both sigmoid and softmax regression suffer from class imbalance
 - the math of three kinds of metrics for classification:
 - Confusion Matrix
 - Accuracy
 - Precision, Recall and F1-Score

Kaggle Competition: Predicting Breast Cancer



Figure 1: Kaggle competition: predicting breast cancer. Picture courtesy of Kaggle.

- Breast Cancer Wisconsin (Diagnostic) dataset:

- features: ID number + 30 variables computed from a digitized image of a fine needle aspirate (FNA) of a breast mass, describing characteristics of the cell nuclei present in the image
- target: the diagnosis of breast cancer, Benign (B) or Malignant (M)

Kaggle Competition: Digit Recognizer



Figure 2: Kaggle competition: digit recognizer. Picture courtesy of Kaggle.

● Modified National Institute of Standards and Technology (MNIST) dataset:

- features: flattened 28×28 (i.e., 784) pixels (taking value in $[0, 255]$) in the image of a digit
- target: the digit in each image, taking value in $[0, 9]$

Classification

- The goal of the two Kaggle competitions is using the features to predict the target.
- The target in each dataset is a *Discrete* variable, since it can only take finite number of values:
 - Breast Cancer Wisconsin: Benign or Malignant
 - MNIST: $[0, 9]$
- We call this kind of prediction (where the target is a discrete variable) *Classification*.
- We will apply a simple classification model (a.k.a., *Classifier*), *Logistic Regression*, to the two competitions.

Three Types of Classification

- We can divide classification into three categories:
 - *Binary Classification*
 - the target has 2 classes and a sample can only belong to 1 class
 - e.g., the breast cancer diagnosis can either be benign or malignant
 - *Multiclass Classification*
 - the target has >2 two classes and a sample can only belong to 1 class
 - e.g., a digit can take any value between 0 and 9
 - *Multilabel Classification*
 - the target has ≥ 2 two classes and a sample can belong to any number of the classes at the same time
 - e.g., an image can contain people, dog, frisbee, sky and meadow at the same time
- Here we will focus on binary classification and multiclass classification.
- We will discuss multilabel classification in [/p3_c2_s3_convolutional_neural_networks](#).

Logistic Regression

- Depending on the type of classification (binary or multiclass), logistic regression can be divided into two categories:
 - Sigmoid Regression for binary classification
 - Softmax Regression for multiclass classification
- The two methods are similar in that, they both:
 - model the probability distribution of the classes
 - use Maximum Likelihood Estimation to estimate the parameters
 - pick the class with the highest probability as the predicted class
- The two methods are different in that:
 - sigmoid regression uses the Sigmoid function (hence the name) to model the probability distribution, which in the binary case is Bernoulli distribution
 - softmax regression uses the Softmax function (hence the name) to model the probability distribution, which in the multiclass case is Categorical distribution

Bernoulli Distribution

- We use *Sigmoid Regression* (a.k.a., *Binomial Logistic Regression*) for binary classification, where the probability distribution of a binary target in a sample, $p(y|\boldsymbol{\theta})$, follows a bernoulli distribution:

$$p(y|\boldsymbol{\theta}) = \begin{cases} p & \text{if } y = 1, \\ 1 - p & \text{if } y = 0. \end{cases} \quad (1)$$

Here:

- y is the class of the sample, which is either 0 or 1
- $\boldsymbol{\theta}$ is the parameter vector
- Eq. (1) says that:
 - if the class of the sample, y , is 1, then the probability of the class is p
 - if the class of the sample, y , is 0, then the probability of the class is $1 - p$
 - the sum of the probabilities across the two classes is 1
- We can also write eq. (1) as

$$p(y|\boldsymbol{\theta}) = p^y(1 - p)^{1-y}. \quad (2)$$

Mathematical Model

- Sigmoid regression uses a *Sigmoid* function to model the probability of class 1, p :

$$p = \frac{1}{1 + e^{-n}}, \quad (3)$$

where the net input, n , is a weighted sum of features:

$$n = b + w_1x_1 + \cdots + w_nx_n = b + \mathbf{xw} = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^T = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}. \quad (4)$$

Here:

- p is the probability of class 1 of a sample
- n is the net input of a sample
- b is the bias
- $\mathbf{w}$ is the $n \times 1$ weight vector (with n being the number of features):

$$\mathbf{w} = \begin{bmatrix} w_1 \cdots w_n \end{bmatrix}^T \quad (5)$$

- $\mathbf{x}$ is the $1 \times n$ feature vector of a sample (with n being the number of features):

$$\mathbf{x} = \begin{bmatrix} x_1 \cdots x_n \end{bmatrix} \quad (6)$$

- $\boldsymbol{\theta}$ is the $(n + 1) \times 1$ parameter vector:

$$\boldsymbol{\theta} = \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^T \quad (7)$$

Architecture

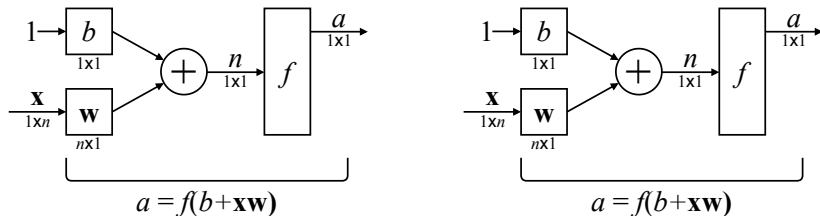


Figure 3: The architecture of linear regression (left) and sigmoid regression (right).

- As fig. 3 shows, a major part of the architecture of linear regression and sigmoid regression are exactly the same:
 - the bias b , net input n and output a for both are scalars
 - the weight $\mathbf{w}$ for both are $n \times 1$ vector
- The only difference between the two architectures is:
 - the activation function in linear regression is identity function:

$$f(n) = n \quad (8)$$

- the activation function in sigmoid regression is sigmoid function:

$$f(n) = \frac{1}{1 + e^{-n}} \quad (9)$$

Sklearn LogisticRegression: Code Example

- See /p2_c2_s3_logistic_regression/code_example/code_example_bcw:
 - 1 cell 58
 - 2 cell 62



Good practice

- As discussed in /p2_c1_data_preprocessing, since sklearn LogisticRegression supports hyperparameter `class_weight`, it is recommended to set the hyperparameter as 'balanced', so as to use cost-based method to address class imbalance.

Classification

- As discussed earlier, we can calculate the probability of class 1, p , using eq. (3)

$$p = \frac{1}{1 + e^{-n}}, \quad (3)$$

where the net input, n , is a weighted sum of features:

$$n = b + w_1x_1 + \cdots + w_nx_n = b + \mathbf{xw} = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}. \quad (4)$$

- Based on eqs. (3) and (4), we can use the following threshold function to predict a new sample's class, $\hat{y}$:

$$\hat{y} = \begin{cases} 1, & \text{if } p \geq 0.5, \\ 0, & \text{if } p < 0.5. \end{cases} \quad (10)$$

- Eq. (10) says that:
 - if the probability of class 1, p , is ≥ 0.5 , then we predict the class of the new sample, $\hat{y}$, as 1
 - if the probability of class 1, p , is < 0.5 (i.e., the probability of class 0, $1 - p$, is ≥ 0.5), then we predict the class of the new sample, $\hat{y}$, as 0
- However, unlike predicting a new sample's class, estimating the parameters, $\boldsymbol{\theta} = [b \ w_1 \cdots w_n]^\top$, is not straightforward.

Binomial Distribution

- As discussed earlier, the probability distribution of a binary target in one sample follows a bernoulli distribution, $p(y|\theta)$, given in eq. (2)

$$p(y|\theta) = p^y (1-p)^{1-y}. \quad (2)$$

- Moreover, the probability distribution of a binary target across m samples follows a *Binomial* distribution:

$$p(y|\theta) = \binom{m}{k} \prod_{i=1}^m (p^i)^{y^i} (1-p^i)^{1-y^i}. \quad (11)$$

Here:

- p^i is the probability of class 1 in sample i
- y^i is the class of sample i , which is either 0 or 1
- m is the number of samples
- k is the sum of y^i across m samples:

$$k = \sum_{i=1}^m y_i \quad (12)$$

- $\binom{m}{k}$ is the *Binomial Coefficient* (hence the name of the distribution), which is the total number of ways to choose an (unordered) subset of k elements from m elements:

$$\binom{m}{k} = \frac{m!}{k!(m-k)!} \quad (13)$$

Maximum Likelihood Estimation

- The binomial distribution is given in eq. (11)

$$\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \binom{m}{k} \prod_{i=1}^m (p^i)^{y^i} (1-p^i)^{1-y^i}, \quad (11)$$

where p is the probability of class 1, given in eq. (3)

$$p = \frac{1}{1 + e^{-n}}, \quad (3)$$

and n is the net input, given in eq. (4)

$$n = b + w_1x_1 + \cdots + w_nx_n = b + \mathbf{xw} = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}. \quad (4)$$

- Since the binomial distribution models the distribution of the data, it is also called the *Likelihood*.
- We can estimate the parameters, $\boldsymbol{\theta} = \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top$, by maximizing the likelihood, $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$, given in eq. (11):

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \binom{m}{k} \prod_{i=1}^m (p^i)^{y^i} (1-p^i)^{1-y^i}. \quad (14)$$

- This optimization is called *Maximum Likelihood Estimation* (MLE hereafter).

Kullback–Leibler Divergence

- To see why MLE makes sense, we need the definition of Kullback–Leibler Divergence.
- Let $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$ be the real distribution of data and $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$ the estimated distribution.
- The *Kullback–Leibler Divergence* (KL-divergence hereafter) from $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$ to $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$, $D_{KL}(\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)||\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}))$, is

$$D_{KL}(\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)||\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})) = E_{\mathbf{y} \sim \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)} \left[\log \frac{\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)}{\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})} \right]. \quad (15)$$

- That is, $D_{KL}(\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)||\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}))$ is the expectation (with respect to $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$) of the logarithmic difference between $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$ and $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$.

- Moreover, $D_{KL}(\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)||\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}))$ takes the minimal value, zero, when $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*) = \mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$. (16)

- See proof of eq. (16) in Appendix (page 84).
- It turn out that, maximizing the likelihood (i.e., MLE) equates minimizing the KL-divergence (see proof of this claim in Appendix, page 85).
- Based on eq. (16), MLE indeed makes sense as it minimizes the difference between $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$ and $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$.
- This is desirable because we want $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$ to be as close to $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}^*)$ as possible.

Binomial Cross Entropy

- As discussed earlier, we can use MLE (eq. (14)) to estimate the parameters, $\theta = [b \quad w_1 \cdots w_n]^\top$:

$$\theta^* = \arg \max_{\theta} p(y|\theta) = \arg \max_{\theta} \binom{m}{k} \prod_{i=1}^m (p^i)^{y^i} (1 - p^i)^{1-y^i}. \quad (14)$$

- It turns out that we can write eq. (14) as

$$\theta^* = \arg \max_{\theta} p(y|\theta) = \arg \min_{\theta} \left(- \sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)) \right). \quad (17)$$

- See the proof of eq. (17) in Appendix (pages 86 and 88).
- The item we want to minimize in eq. (17),

$$- \sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)), \quad (18)$$

is called *Binomial Cross Entropy*, which can be calculated as follows:

- we first calculate the product of each class and its log probability
- we then calculate the sum of the product in step 1 across both classes
- we next calculate the sum of the sum in step 2 across all samples
- we last calculate the additive inverse of the sum in step 3

Loss Function

- With the definition of binomial cross entropy given in eq. (18)

$$-\sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)), \quad (18)$$

we can define the loss function of sigmoid regression.

- It turns out that the loss function of sigmoid regression, $\mathcal{L}(\boldsymbol{\theta})$, is the average binomial cross entropy across m samples:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (19)$$

Here:

- m is the number of samples
- y^i is the class of sample i , which is either 0 or 1
- p^i is the probability of class 1 in sample i :

$$p^i = \frac{1}{1 + e^{-n^i}}, \quad (20)$$

where n^i is the net input of class 1 in sample i :

$$n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (21)$$

- We can estimate the parameters, $\boldsymbol{\theta} = [b \quad w_1 \cdots w_n]^\top$, by minimizing the loss function in eq. (19).

Gradient Descent

- Similar to linear regression (see [/p2_c2_s1_linear_regression](#)), we can also use gradient descent to iteratively update the parameters of sigmoid regression, so as to minimize the loss function, given in eq. (19)

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (19)$$

- Similar to linear regression, we will use Mini-Batch Gradient Descent (MBGD) to update the parameters, since as discussed in [/p2_c2_s1_linear_regression](#):
 - in theory, MBGD reduces to Batch Gradient Descent (BGD) / Stochastic Gradient Descent (SGD) when the mini-batch contains all the samples / only one sample, so that we can slightly tweak the equations for MBGD (with respect to the mini-batch size) to get the equations for BGD and SGD
 - in practice, MBGD is more popular in deep learning

MBGD: Loss

- Eq. (19) measures the average binomial cross entropy across m samples:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (19)$$

- Since MBGD updates the parameters using a mini-batch of training samples, $\mathbf{mb}^j$, the loss function, $\mathcal{L}(\boldsymbol{\theta}^j)$, can be written as:

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (22)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real class of sample i
- p^i is the probability of class 1 in sample i , given in eq. (20):

$$p^i = \frac{1}{1 + e^{-n^i}}, \quad (20)$$

where n^i is the net input of class 1 in sample i , given in eq. (21):

$$n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (21)$$

MBGD: Updating Rule

- As discussed in /p2_c2_s1_linear_regression, the updating rule of MBGD is

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}. \quad (23)$$

Here

- $\boldsymbol{\theta}_k^j$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}^j$
- $\boldsymbol{\theta}_k^{j+1}$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}_{j+1}$
- $\mathbf{g}_k^j$ is a the gradient of the loss, $\mathcal{L}(\boldsymbol{\theta}^j)$:
 - it is the first-order derivative of $\mathcal{L}(\boldsymbol{\theta}^j)$, with respect to the parameters in epoch k , updated using mini-batch $\mathbf{mb}^j$, $\boldsymbol{\theta}_k^j$
 - it is also the direction that leads to the steepest ascent of $\mathcal{L}(\boldsymbol{\theta}^j)$
- η_k is the learning rate that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k^j$ along $\mathbf{g}_k^j$ to search for $\boldsymbol{\theta}_k^{j+1}$)

MBGD: Updating Rule

- By deriving the gradient, $\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}$, eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j} \quad (23)$$

can be written as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix}^\top (\mathbf{y}^j - \mathbf{p}^j). \quad (24)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real class of sample i
- p^i is the probability of class 1 in sample i :

$$p^i = \frac{1}{1 + e^{-n^i}} \quad \text{where} \quad n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta}^j \quad (25)$$

- $\mathbf{y}^j$ is the $|\mathbf{mb}^j| \times 1$ real class vector of $\mathbf{mb}^j$
- $\mathbf{p}^j$ is the $|\mathbf{mb}^j| \times 1$ probability vector of class 1 of $\mathbf{mb}^j$

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix} \boldsymbol{\theta}^j \quad (26)$$

- $\mathbf{x}^i$ is the $1 \times |\mathbf{mb}^j|$ feature vector of sample i , and $\mathbf{X}^j$ the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$

- See the proof of eq. (24) in Appendix (pages 89 to 98).

MBGD: The Implementation

- See [/p2_c2_s3_logistic_regression/code_example/code_example_bcw:](#)
 - ① cell 58
 - ② cell 63
- See [/models/p2_shallow_learning:](#)
 - ① cell 5



Good practice

- It is recommended to use small batches (from 2 to 32) in MBGD [Masters and Luschi, 2018].

MBGD + Lasso: Loss

- With the MBGD loss (second item in eq. (27)) and the regularization term of lasso (third item), the loss of MBGD + lasso is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)}_{\text{MBGD + lasso loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i))}_{\text{MBGD loss}} + \underbrace{\alpha \sum_{j=1}^n |w_j|}_{\text{lasso term}}. \quad (27)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real class of sample i
- p^i is the probability of class 1 in sample i , given in eq. (20):

$$p^i = \frac{1}{1 + e^{-n^i}}, \quad (20)$$

where n^i is the net input of class 1 in sample i , given in eq. (21):

$$n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (21)$$

- α is the regularization parameter

MBGD + Lasso: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (22)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (22)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (27)), with MBGD + lasso loss, $\mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)$ (first item in eq. (27)), we can write the updating rule of MBGD + lasso as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (28)$$

MBGD + Lasso: Updating Rule

- By deriving the gradient in eq. (28), $\nabla \mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)^\top|_{\boldsymbol{\theta}^j=\boldsymbol{\theta}_k^j}$, we can write eq. (28) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \mathbf{p}^j) - \alpha [0 \quad \text{sgn}(w_1) \cdots \text{sgn}(w_n)]^\top \right). \quad (29)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{y}^j$ is the $|\mathbf{mb}^j| \times 1$ real class vector of $\mathbf{mb}^j$
- $\mathbf{p}^j$ is the $|\mathbf{mb}^j| \times 1$ probability vector of class 1 of $\mathbf{mb}^j$, given in eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = [1 \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (26)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- sgn is the *Sign* function:

$$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases} \quad (30)$$

- The proof of eq. (29) is similar to that in Appendix (pages 89 to 98) and that in Appendix in [/p2.c2.s1.linear_regression.](#)

MBGD + Lasso: The Implementation

- See [/models/p2_shallow_learning:](#)
 - ① cell 5

MBGD + Ridge: Loss

- With the MBGD loss (second item in eq. (31)) and the regularization term of ridge (third item), the loss of MBGD + ridge is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)}_{\text{MBGD + ridge loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i))}_{\text{MBGD loss}} + \underbrace{\frac{\alpha}{2} \sum_{j=1}^n w_j^2}_{\text{ridge term}}. \quad (31)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real class of sample i
- p^i is the probability of class 1 in sample i , given in eq. (20):

$$p^i = \frac{1}{1 + e^{-n^i}}, \quad (20)$$

where n^i is the net input of class 1 in sample i , given in eq. (21):

$$n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (21)$$

- α is the regularization parameter

MBGD + Ridge: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (22)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (22)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (31)), with MBGD + ridge loss, $\mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)$ (first item in eq. (31)), we can write the updating rule of MBGD + lasso as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (32)$$

MBGD + Ridge: Updating Rule

- By deriving the gradient in eq. (32), $\nabla \mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}$, we can write eq. (32) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} [\mathbf{1} \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \mathbf{p}^j) - \alpha [0 \quad w_1 \cdots w_n]^\top \right). \quad (33)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{y}^j$ is the $|\mathbf{mb}^j| \times 1$ real class vector of $\mathbf{mb}^j$
- $\mathbf{p}^j$ is the $|\mathbf{mb}^j| \times 1$ probability vector of class 1 of $\mathbf{mb}^j$, given in eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (26)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- α is the regularization parameter
- The proof of eq. (29) is similar to that in Appendix (pages 89 to 98) and that in Appendix in [/p2_c2_s1_linear_regression](#).

MBGD + Ridge: The Implementation

- See [/models/p2_shallow_learning:](#)
 - 1 cell 5

MBGD + Elastic Net: Loss

- With the MBGD loss (second item in eq. (68)) and the regularization term of elastic net (third item), the loss of MBGD + elastic net is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)}_{\text{MBGD + elastic net loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y^i \log(p^i) + (1 - y^i) \log(1 - p^i) \right)}_{\text{MBGD loss}} + \underbrace{\alpha\gamma \sum_{j=1}^n |w_j| + \frac{\alpha(1-\gamma)}{2} \sum_{j=1}^n w_j^2}_{\text{elastic net term}}. \quad (34)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real class of sample i
- p^i is the probability of class 1 in sample i , given in eq. (20):

$$p^i = \frac{1}{1 + e^{-n^i}}, \quad (20)$$

where n^i is the net input of class 1 in sample i , given in eq. (21):

$$n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (21)$$

- α and γ are the regularization parameters

MBGD + Elastic Net: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (22)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (22)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (68)), with MBGD + elastic net loss, $\mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)$ (first item in eq. (68)), we can write the updating rule of MBGD + elastic net as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (35)$$

MBGD + Elastic Net: Updating Rule

- By deriving the gradient in eq. (69), $\nabla \mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}$, we can write eq. (69) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \mathbf{p}^j) - \alpha \gamma [0 \quad \text{sgn}(w_1) \cdots \text{sgn}(w_n)]^\top - \alpha(1 - \gamma) [0 \quad w_1 \cdots w_n]^\top \right). \quad (36)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{y}^j$ is the $|\mathbf{mb}^j| \times 1$ real class vector of $\mathbf{mb}^j$
- $\mathbf{p}^j$ is the $|\mathbf{mb}^j| \times 1$ probability vector of class 1 of $\mathbf{mb}^j$, given in eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 x_1^j + \cdots + w_n x_n^j = [1 \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (26)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- sgn is the *Sign* function:

$$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases} \quad (37)$$

- The proof of eq. (29) is similar to that in Appendix (pages 89 to 98) and that in Appendix in [/p2_c2_s1_linear_regression](#).

MBGD + Elastic Net: The Implementation

- See [/models/p2_shallow_learning:](#)
 - ① cell 5

Categorical Distribution

- We use *Softmax Regression* (a.k.a., *Multinomial Logistic Regression*) for multiclass classification, where the probability distribution of a categorical target in a sample, $p(y|\theta)$, follows a categorical distribution:

$$p(y|\theta) = \begin{cases} p_1 & \text{if } y_1 = 1, \\ \vdots & \vdots \\ p_c & \text{if } y_c = 1. \end{cases} \quad (38)$$

Here:

- c is the number of unique classes of the categorical target
- y is the class of the sample, with:
 - $y_k = 1$ (where $1 \leq k \leq c$) says that the sample belongs to class k
 - $y_k = 0$ (where $1 \leq k \leq c$) says that the sample belongs to other classes
- p_k (where $1 \leq k \leq c$) is the probability of the sample belonging to class k , where the sum of the probabilities across the c classes is 1:

$$\sum_{k=1}^c p_k = 1 \quad (39)$$

- θ is the parameter vector
- We can also write eq. (38) as

$$p(y|\theta) = \prod_{k=1}^c p_k^{y_k}. \quad (40)$$

Mathematical Model

- Softmax regression uses a *Softmax* function to model the probability distribution of all the possible classes in a sample, $\mathbf{p}$:

$$\mathbf{p} = \frac{e^{\mathbf{n}}}{\sum e^{\mathbf{n}}}, \quad (41)$$

where the net input vector, $\mathbf{n}$, is a vector of weighted sum of features:

$$\mathbf{n} = \mathbf{b} + \mathbf{w}_1 x_1 + \cdots + \mathbf{w}_n x_n = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} \mathbf{b} & \mathbf{w}_1 \cdots \mathbf{w}_n \end{bmatrix}^T = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}. \quad (42)$$

Here:

- $x_1, \dots, x_n$ are the features of a sample
- $\mathbf{b}$ is a $1 \times c$ bias vector (where c is the number of unique classes of the target)
- $\mathbf{w}_1, \dots, \mathbf{w}_n$ are $1 \times c$ weight vectors of $x_1, \dots, x_n$
- $\mathbf{x}$ is a $1 \times n$ feature vector of a sample (with n being the number of features):

$$\mathbf{x} = [x_1 \cdots x_n] \quad (43)$$

- $\boldsymbol{\theta}$ is a $(n+1) \times c$ parameter matrix:

$$\boldsymbol{\theta} = \begin{bmatrix} \mathbf{b} & \mathbf{w}_1 \cdots \mathbf{w}_n \end{bmatrix}^T \quad (44)$$

Architecture

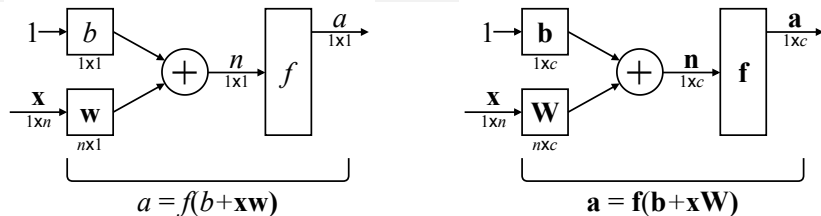


Figure 4: The architecture of sigmoid regression (left) and softmax regression (right).

- As fig. 4 shows, a major part of the architecture of sigmoid regression and softmax regression are different:
 - the bias, net input and output are:
 - scalars (b , n and a) in sigmoid regression
 - $1 \times c$ vectors ($\mathbf{b}$, $\mathbf{n}$ and $\mathbf{a}$) in softmax regression
 - the weights are:
 - scalars ($w_1, \dots, w_n$) in sigmoid regression
 - $1 \times c$ vectors ($\mathbf{w}_1, \dots, \mathbf{w}_n$) in softmax regression
 - the activation is:
 - sigmoid (f) in sigmoid regression
 - softmax ($\mathbf{f}$) in softmax regression

Sklearn LogisticRegression: Code Example

- See /p2_c2_s3_logistic_regression/code_example/code_example_bcw:
 - 1 cell 58
 - 2 cell 62



Good practice

- As discussed in /p2_c1_data_preprocessing, since sklearn LogisticRegression supports hyperparameter `class_weight`, it is recommended to set the hyperparameter as 'balanced', so as to use cost-based method to address class imbalance.

Classification

- As discussed earlier, we can calculate the probability distribution of all the possible classes, $\mathbf{p}$, using eq. (41)

$$\mathbf{p} = \frac{e^{\mathbf{n}}}{\sum e^{\mathbf{n}}}, \quad (41)$$

where the net input vector, $\mathbf{n}$, is a vector of weighted sum of features:

$$\mathbf{n} = \mathbf{b} + \mathbf{w}_1 x_1 + \cdots + \mathbf{w}_n x_n = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} \mathbf{b} & \mathbf{w}_1 \cdots \mathbf{w}_n \end{bmatrix}^T = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}. \quad (42)$$

- Based on eqs. (41) and (42), we have the probability of each possible class k , p_k :

$$p_k = \frac{e^{n_k}}{\sum_{k=1}^c e^{n_k}}, \quad (45)$$

where the net input of class k , n_k , is a weighted sum of features:

$$n_k = b^k + w_1^k x_1 + \cdots + w_n^k x_n = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \begin{bmatrix} b^k & w_1^k \cdots w_n^k \end{bmatrix}^T = \begin{bmatrix} 1 & \mathbf{x} \end{bmatrix} \boldsymbol{\theta}^k. \quad (46)$$

- Based on eqs. (45) and (46), we can use the following function to predict a new sample's class, $\hat{y}$:

$$\hat{y} = \arg \max_k p_k. \quad (47)$$

- Eq. (47) says that, the predicted class, $\hat{y}$, is the class that has the highest probability across all the classes.
- However, unlike predicting a new sample's class, estimating the parameters, $\boldsymbol{\theta} = \begin{bmatrix} \mathbf{b} & \mathbf{w}_1 \cdots \mathbf{w}_n \end{bmatrix}^T$, is not straightforward.

Multinomial Distribution

- As discussed earlier, the probability distribution of a categorical target in one sample follows a categorical distribution, $p(y|\theta)$, given in eq. (40)

$$p(y|\theta) = \prod_{k=1}^c p_k^{y_k}. \quad (40)$$

- Similarly, the probability distribution of a categorical target across m samples follows a *Multinomial* distribution:

$$p(y|\theta) = \binom{n}{n_1, \dots, n_c} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}. \quad (48)$$

Here:

- p_k^i is the probability of class k in sample i
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- m is the number of samples
- n_k (where $1 \leq k \leq c$) is the sum of y_k^i across m samples and $n = \sum_{k=1}^c n_k$
- $\binom{n}{n_1, \dots, n_c}$ is the *Multinomial Coefficient* (hence the name of the distribution), which is the number of combinations of $n_1, \dots, n_c$ such that $\sum_{k=1}^c n_k = n$:

$$\binom{n}{n_1, \dots, n_c} = \frac{n!}{\prod_{k=1}^c n_k!} \quad (49)$$

Maximum Likelihood Estimation

- The multinomial distribution is given in eq. (48)

$$\mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \binom{n}{n_1, \dots, n_c} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}, \quad (48)$$

where p_k^i is the probability of class k in sample i ,

$$p_k^i = \frac{e^{n_k^i}}{\sum_{l=1}^c e^{n_l^i}}, \quad (50)$$

and n_k^i is the net input of class k in sample i :

$$n_k^i = b^k + w_1^k x_1^i + \dots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] [b^k \quad w_1^k \dots w_n^k]^\top = [1 \quad \mathbf{x}^i] \boldsymbol{\theta}^k. \quad (51)$$

- Similar to binomial distribution, since multinomial distribution models the distribution of the data, it is also called the likelihood.
- Similar to binomial distribution, we can also use MLE to estimate the parameters, $\boldsymbol{\theta} = [\mathbf{b} \quad \mathbf{w}_1 \dots \mathbf{w}_n]^\top$, by maximizing the likelihood, $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$, given in eq. (48):

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \binom{n}{n_1, \dots, n_c} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}. \quad (52)$$

- As discussed earlier, maximizing the likelihood makes sense since it equates minimizing the KL-divergence (see proof of this claim in Appendix, page 84).

Multinomial Cross Entropy

- As discussed earlier, we can use MLE (eq. (52)) to estimate the parameters, $\theta = [\mathbf{b} \quad \mathbf{w}_1 \cdots \mathbf{w}_n]^\top$:

$$\theta^* = \arg \max_{\theta} \mathbf{p}(\mathbf{y}|\theta) = \arg \max_{\theta} \binom{n}{n_1, \dots, n_c} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}. \quad (52)$$

- It turns out that we can write eq. (52) as

$$\theta^* = \arg \max_{\theta} \mathbf{p}(\mathbf{y}|\theta) = \arg \min_{\theta} \left(- \sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i) \right). \quad (53)$$

- See the proof of eq. (53) in Appendix (pages 99 and 101).
- The item we want to minimize in eq. (53),

$$- \sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i), \quad (54)$$

is called *Multinomial Cross Entropy*, which can be calculated as follows:

- ① we first calculate the product of each class and its log probability
- ② we then calculate the sum of the product in step 1 across all classes
- ③ we next calculate the sum of the sum in step 2 across all samples
- ④ we last calculate the additive inverse of the sum in step 3

Loss Function

- With the definition of multinomial cross entropy given in eq. (54)

$$-\sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i), \quad (54)$$

we can define the loss function of softmax regression.

- It turns out that the loss function of softmax regression, $\mathcal{L}(\boldsymbol{\theta})$, is the average multinomial cross entropy across m samples:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i) \quad \text{where :} \quad (55)$$

- m is the number of samples
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- p_k^i is the probability of class k in sample i , given in eq. (50):

$$p_k^i = \frac{e^{n_k^i}}{\sum_{l=1}^c e^{n_l^i}}, \quad (50)$$

where n_k^i is the net input of class k in sample i , given in eq. (51):

$$n_k^i = b^k + w_1^k x_1^i + \cdots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (51)$$

- We can estimate the parameters, $\boldsymbol{\theta} = [\mathbf{b} \quad \mathbf{w}_1 \cdots \mathbf{w}_n]^\top$, by minimizing the loss function in eq. (55).

Gradient Descent

- Similar to sigmoid regression, we can also use gradient descent to iteratively update the parameters of softmax regression, so as to minimize the loss function, given in eq. (55)

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i). \quad (55)$$

- Similar to sigmoid regression, we will use Mini-Batch Gradient Descent (MBGD) to update the parameters, since as discussed in [/p2_c2_s1_linear_regression:](#)
 - in theory, MBGD reduces to Batch Gradient Descent (BGD) / Stochastic Gradient Descent (SGD) when the mini-batch contains all the samples / only one sample, so that we can slightly tweak the equations for MBGD (with respect to the mini-batch size) to get the equations for BGD and SGD
 - in practice, MBGD is more popular in deep learning

MBGD: Loss

- Eq. (55) measures the average multinomial cross entropy across m samples:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^c y_k^i \log(p_k^i). \quad (55)$$

- Since MBGD updates the parameters using a mini-batch of training samples, $\mathbf{mb}^j$, the loss function, $\mathcal{L}(\boldsymbol{\theta}^j)$, can be written as:

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i) \quad \text{where :} \quad (56)$$

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- p_k^i is the probability of class k in sample i , given in eq. (50):

$$p_k^i = \frac{e^{n_k^i}}{\sum_{l=1}^c e^{n_l^i}}, \quad (50)$$

where n_k^i is the net input of class k in sample i , given in eq. (51):

$$n_k^i = b^k + w_1^k x_1^i + \cdots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (51)$$

MBGD: Updating Rule

- As discussed earlier, the updating rule of MBGD is given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}. \quad (23)$$

Here

- $\boldsymbol{\theta}_k^j$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}^j$
- $\boldsymbol{\theta}_k^{j+1}$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}_{j+1}$
- $\mathbf{g}_k^j$ is a the gradient of the loss, $\mathcal{L}(\boldsymbol{\theta}^j)$:
 - it is the first-order derivative of $\mathcal{L}(\boldsymbol{\theta}^j)$, with respect to the parameters in epoch k , updated using mini-batch $\mathbf{mb}^j$, $\boldsymbol{\theta}_k^j$
 - it is also the direction that leads to the steepest ascent of $\mathcal{L}(\boldsymbol{\theta}^j)$
- η_k is the learning rate that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k^j$ along $\mathbf{g}_k^j$ to search for $\boldsymbol{\theta}_k^{j+1}$)

MBGD: Updating Rule

- By deriving the gradient, $\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}$, eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j} \quad (23)$$

can be written as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} [\mathbf{1} \quad \mathbf{X}^j]^\top (\mathbf{Y}^j - \mathbf{P}^j) \quad \text{where :} \quad (57)$$

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{Y}^j$ is the $|\mathbf{mb}^j| \times c$ indicator matrix of $\mathbf{mb}^j$
- $\mathbf{P}^j$ is the $|\mathbf{mb}^j| \times c$ probability matrix of $\mathbf{mb}^j$

$$\mathbf{P}^j = \frac{e^{\mathbf{N}^j}}{\sum e^{\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1}\mathbf{b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (58)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- See the proof of eq. (57) in Appendix (pages 102 and 107).

MBGD: The Implementation

- See /p2_c2_s3_logistic_regression/code_example/code_example_mnist:
 - ① cell 53
 - ② cell 58
- See /models/p2_shallow_learning:
 - ① cell 5



Good practice

- It is recommended to use small batches (from 2 to 32) in MBGD [Masters and Luschi, 2018].

MBGD + Lasso: Loss

- With the MBGD loss (second item in eq. (59)) and the regularization term of lasso (third item), the loss of MBGD + lasso is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)}_{\text{MBGD + lasso loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i)}_{\text{MBGD loss}} + \underbrace{\alpha \sum_{j=1}^n |w_j|}_{\text{lasso term}}. \quad (59)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- p_k^i is the probability of class k in sample i , given in eq. (50):

$$p_k^i = \frac{e^{n_k^i}}{\sum_{l=1}^c e^{n_l^i}}, \quad (50)$$

where n_k^i is the net input of class k in sample i , given in eq. (51):

$$n_k^i = b^k + w_1^k x_1^i + \cdots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (51)$$

- α is the regularization parameter

MBGD + Lasso: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (56)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i). \quad (56)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (59)), with MBGD + lasso loss, $\mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)$ (first item in eq. (59)), we can write the updating rule of MBGD + lasso as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (60)$$

MBGD + Lasso: Updating Rule

- By deriving the gradient in eq. (60), $\nabla \mathcal{L}_{m+l_1}(\boldsymbol{\theta}^j)^\top|_{\boldsymbol{\theta}^j=\boldsymbol{\theta}_k^j}$, we can write eq. (60) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix}^\top (\mathbf{Y}^j - \mathbf{P}^j) - \alpha \begin{bmatrix} 0 & \text{sgn}(w_1) \cdots \text{sgn}(w_n) \end{bmatrix}^\top \right). \quad (61)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{Y}^j$ is the $|\mathbf{mb}^j| \times c$ indicator matrix of $\mathbf{mb}^j$
- $\mathbf{P}^j$ is the $|\mathbf{mb}^j| \times c$ probability matrix of $\mathbf{mb}^j$

$$\mathbf{P}^j = \frac{1}{1 + e^{-\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix} \boldsymbol{\theta}^j \quad (62)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- sgn is the *Sign* function:

$$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases} \quad (63)$$

- The proof of eq. (61) is similar to that in Appendix (pages 102 and 107) and that in Appendix in [/p2_c2_s1_linear_regression](#).

MBGD + Lasso: The Implementation

- See [/models/p2_shallow_learning:](#)
 - ① cell 5

MBGD + Ridge: Loss

- With the MBGD loss (second item in eq. (64)) and the regularization term of ridge (third item), the loss of MBGD + ridge is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)}_{\text{MBGD + ridge loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i)}_{\text{MBGD loss}} + \underbrace{\frac{\alpha}{2} \sum_{j=1}^n w_j^2}_{\text{ridge term}}. \quad (64)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- p_k^i is the probability of class k in sample i , given in eq. (50):

$$p_k^i = \frac{1}{1 + e^{-n_k^i}}, \quad (50)$$

where n_k^i is the net input of class k in sample i , given in eq. (51):

$$n_k^i = b^k + w_1^k x_1^i + \cdots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (51)$$

- α is the regularization parameter

MBGD + Ridge: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (56)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i). \quad (56)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (64)), with MBGD + ridge loss, $\mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)$ (first item in eq. (64)), we can write the updating rule of MBGD + lasso as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (65)$$

MBGD + Ridge: Updating Rule

- By deriving the gradient in eq. (65), $\nabla \mathcal{L}_{m+l_2}(\boldsymbol{\theta}^j)^\top|_{\boldsymbol{\theta}^j=\boldsymbol{\theta}_k^j}$, we can write eq. (65) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} [\mathbf{1} \quad \mathbf{X}^j]^\top (\mathbf{Y}^j - \mathbf{P}^j) - \alpha [0 \quad w_1 \cdots w_n]^\top \right). \quad (66)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{Y}^j$ is the $|\mathbf{mb}^j| \times c$ indicator matrix of $\mathbf{mb}^j$
- $\mathbf{P}^j$ is the $|\mathbf{mb}^j| \times c$ probability matrix of $\mathbf{mb}^j$

$$\mathbf{P}^j = \frac{1}{1 + e^{-\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (67)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- The proof of eq. (61) is similar to that in Appendix (pages 102 and 107) and that in Appendix in /p2_c2_s1_linear_regression.

MBGD + Ridge: The Implementation

- See /models/p2_shallow_learning:
 - 1 cell 5

MBGD + Elastic Net: Loss

- With the MBGD loss (second item in eq. (68)) and the regularization term of elastic net (third item), the loss of MBGD + elastic net is the sum of the two:

$$\underbrace{\mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)}_{\text{MBGD + elastic net loss}} = \underbrace{-\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i)}_{\text{MBGD loss}} + \underbrace{\alpha\gamma \sum_{j=1}^n |w_j| + \frac{\alpha(1-\gamma)}{2} \sum_{j=1}^n w_j^2}_{\text{elastic net term}}. \quad (68)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $y_k^i = 1$ (where $1 \leq k \leq c$) says that sample i belongs to class k
- $y_k^i = 0$ (where $1 \leq k \leq c$) says that sample i belongs to other classes
- p_k^i is the probability of class k in sample i , given in eq. (50):

$$p_k^i = \frac{1}{1 + e^{-n_k^i}}, \quad (50)$$

where n_k^i is the net input of class k in sample i , given in eq. (51):

$$n_k^i = b^k + w_1^k x_1^i + \cdots + w_n^k x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta} \quad (51)$$

- α and γ are the regularization parameters

MBGD + Elastic Net: Updating Rule

- The updating rule of MBGD was given in eq. (23)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}, \quad (23)$$

where the MBGD loss, $\mathcal{L}(\boldsymbol{\theta}^j)$, was given in eq. (56)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{k=1}^c y_k^i \log(p_k^i). \quad (56)$$

- By replacing the MBGD loss in eq. (23), $\mathcal{L}(\boldsymbol{\theta}^j)$ (also the second item in eq. (68)), with MBGD + elastic net loss, $\mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)$ (first item in eq. (68)), we can write the updating rule of MBGD + elastic net as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k^j - \eta_k \nabla \mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}. \quad (69)$$

MBGD + Elastic Net: Updating Rule

- By deriving the gradient in eq. (69), $\nabla \mathcal{L}_{m+l_{12}}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}^j = \boldsymbol{\theta}_k^j}$, we can write eq. (69) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \left(\frac{1}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{Y}^j - \mathbf{P}^j) - \alpha\gamma [0 \quad \text{sgn}(w_1) \cdots \text{sgn}(w_n)]^\top - \alpha(1-\gamma) [0 \quad w_1 \cdots w_n]^\top \right). \quad (70)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{Y}^j$ is the $|\mathbf{mb}^j| \times c$ indicator matrix of $\mathbf{mb}^j$
- $\mathbf{P}^j$ is the $|\mathbf{mb}^j| \times c$ probability matrix of $\mathbf{mb}^j$

$$\mathbf{P}^j = \frac{1}{1 + e^{-\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = [1 \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (71)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- sgn is the *Sign* function:

$$\text{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases} \quad (72)$$

- The proof of eq. (61) is similar to that in Appendix (pages 102 to 107) and that in Appendix in [/p2_c2_s1_linear_regression](#).

MBGD + Elastic Net: The Implementation

- See [/models/p2_shallow_learning:](#)
 - 1 cell 5

Class Imbalance: Revisit

Table 1: The first 5 features and target (diagnosis) of Breast Cancer Wisconsin dataset.

id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
842302	M	17.99	10.38	122.80	1001.0
842517	M	20.57	17.77	132.90	1326.0
84300903	M	19.69	21.25	130.00	1203.0
84348301	M	11.42	20.38	77.58	386.1
84358402	M	20.29	14.34	135.10	1297.0

- As discussed in [/p2_c1_data_preprocessing](#), a key challenge in classification is *Class Imbalance*, where the number of samples for one class is higher than the number for the other classes.
- We usually call the class with more samples the *Majority Class*, whereas the class with fewer samples the *Minority Class*.
- Take the target, diagnosis, in table 1 for example:
 - majority class: benign (with 357 or 63% samples)
 - minority class: malignant (with 212 or 37% samples)

The Problem of Class Imbalance

- The problem of class imbalance is that, when training a model from imbalanced data such as Breast Cancer Wisconsin (table 1), the model will have a bias against the minority class (malignant):
 - that is, even before looking at any clinical or pathological status of a patient, the model already decides that it is less likely (or, in the extreme case, impossible) for the patient to have malignant cancer cells
 - however, wrongly predicting cancer cells as benign when they are actually malignant could lead to lethal consequences
- Now let us use the updating rule of sigmoid and softmax regression derived in Appendix (pages 89 to 98 and pages 102 to 107) to explain where the bias against the minority class comes from.

The Bias in Sigmoid Regression

- The updating rule of sigmoid regression is shown in eq. (24)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} [\mathbf{1} \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \mathbf{p}^j). \quad (24)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{y}^j$ is the $|\mathbf{mb}^j| \times 1$ real class vector of $\mathbf{mb}^j$
- $\mathbf{p}^j$ is the $|\mathbf{mb}^j| \times 1$ probability vector of class 1 of $\mathbf{mb}^j$, given in eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (26)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- Since $\boldsymbol{\theta} = [b \quad w_1 \dots w_n]^\top$, then based on eq. (24), we have

$$b_k^{j+1} = b_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \mathbf{1}^\top (\mathbf{y}^j - \mathbf{p}^j). \quad (73)$$

The Bias in Sigmoid Regression

- Based on eq. (73)

$$b_k^{j+1} = b_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \mathbf{1}^\top (\mathbf{y}^j - \mathbf{p}^j), \quad (73)$$

if class 1 is the majority class:

- it is more likely that the samples in mini-batch $\mathbf{mb}^j$ belong to class 1 (i.e., it is more likely that $\mathbf{y}^j$ is 1)
- since the probability of class 1, $\mathbf{p}^j \in (0, 1)$, it is more likely that $\mathbf{y}^j - \mathbf{p}^j$ is positive
- based on eq. (73), it is more likely that b will increase
- based on eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j, \quad (26)$$

when b increases

- $\mathbf{n}$ (the net input of class 1) increases
- $\mathbf{p}$ (the probability of class 1) increases
- $1 - \mathbf{p}$ (the probability of class 0) decreases
- as a result, the bias against class 0 (the minority class) arises

The Bias in Sigmoid Regression

- Based on eq. (73)

$$b_k^{j+1} = b_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \mathbf{1}^\top (\mathbf{y}^j - \mathbf{p}^j), \quad (73)$$

if class 1 is the minority class:

- it is less likely that the samples in mini-batch $\mathbf{mb}^j$ belong to class 1 (i.e., it is less likely that $\mathbf{y}^j$ is 1)
- since the probability of class 1, $\mathbf{p}^j \in (0, 1)$, it is less likely that $\mathbf{y}^j - \mathbf{p}^j$ is positive
- based on eq. (73), it is less likely that b will increase
- based on eq. (26)

$$\mathbf{p}^j = \frac{1}{1 + e^{-\mathbf{n}^j}} \quad \text{where} \quad \mathbf{n}^j = b + w_1 \mathbf{x}_1^j + \cdots + w_n \mathbf{x}_n^j = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j, \quad (26)$$

when b decreases

- $\mathbf{n}$ (the net input of class 1) decreases
- $\mathbf{p}$ (the probability of class 1) decreases
- $1 - \mathbf{p}$ (the probability of class 0) increases
- as a result, the bias against class 1 (the minority class) arises

The Bias in Softmax Regression

- The updating rule of softmax regression is shown in eq. (57)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} [\mathbf{1} \quad \mathbf{X}^j]^\top (\mathbf{Y}^j - \mathbf{P}^j) \quad \text{where :} \quad (57)$$

Here

- η_k is the learning rate in epoch k
- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $\mathbf{Y}^j$ is the $|\mathbf{mb}^j| \times c$ indicator matrix of $\mathbf{mb}^j$
- $\mathbf{P}^j$ is the $|\mathbf{mb}^j| \times c$ probability matrix of $\mathbf{mb}^j$, given in eq. (71)

$$\mathbf{P}^j = \frac{e^{\mathbf{N}^j}}{\sum e^{\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1}\mathbf{b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (71)$$

- $\mathbf{X}^j$ is the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- Since $\boldsymbol{\theta} = [\mathbf{b} \quad \mathbf{w}_1 \dots \mathbf{w}_n]^\top$, then based on eq. (57), we have

$$\mathbf{b}_k^{j+1} = \mathbf{b}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \mathbf{1}^\top (\mathbf{Y}^j - \mathbf{P}^j). \quad (74)$$

The Bias in Sigmoid Regression

- Based on eq. (74)

$$\mathbf{b}_k^{j+1} = \mathbf{b}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \mathbf{1}^\top (\mathbf{Y}^j - \mathbf{P}^j), \quad (73)$$

if class l is the majority class:

- it is more likely that the samples in mini-batch $\mathbf{mb}^j$ belong to class l (i.e., it is more likely that y_l^j is 1)
- since the probability of class l , $p_l^j \in (0, 1)$, it is more likely that $y_l^j - p_l^j$ is positive
- based on eq. (73), it is more likely that b_l will increase
- based on eq. (71)

$$\mathbf{P}^j = \frac{e^{\mathbf{N}^j}}{\sum e^{\mathbf{N}^j}} \quad \text{where} \quad \mathbf{N}^j = \mathbf{1}\mathbf{b} + \mathbf{x}_1^j \mathbf{w}_1 + \cdots + \mathbf{x}_n^j \mathbf{w}_n = [\mathbf{1} \quad \mathbf{X}^j] \boldsymbol{\theta}^j \quad (71)$$

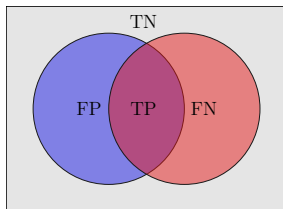
when b_l increases

- n_l (the net input of class l) increases
- p_l (the probability of class l) increases
- $1 - p_l$ (the probability of the other classes) decreases
- as a result, the bias against the other classes (the minority classes) arises

Metrics for Classification

- Unlike regression which uses Mean Squared Error to measure their performance, classification uses a different set of metrics.
- Here we will focus on three kinds of metrics:
 - Confusion Matrix
 - Accuracy
 - Precision, Recall and F1-score
- For illustration purposes, we will use binary classification (with classes Positive and Negative) to explain these measurements.

Confusion Matrix



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

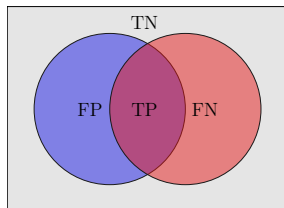
Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

- Based on the TP, FP, TN and FN in fig. 5, we have

$$P = TP + FN, \quad N = TN + FP, \quad \hat{P} = TP + FP, \quad \hat{N} = TN + FN. \quad (75)$$

- Given the confusion matrix, we can obtain the remaining two kinds of metrics:
 - accuracy
 - precision, recall and F1-score

Accuracy



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

- Based on the TP, FP, TN and FN in fig. 5, *Accuracy*, acc , is defined as follows:

$$acc = \frac{TP + TN}{TP + TN + FP + FN}. \quad (76)$$

The Problem of Accuracy

- It is recommended not to use accuracy to measure the performance of a classifier, particularly when the data is imbalanced.
- This can be seen from the definition of accuracy, acc , given in eq. (76)

$$acc = \frac{TP + TN}{TP + TN + FP + FN}. \quad (76)$$

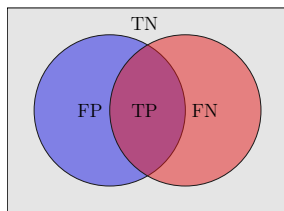
- Concretely, for imbalanced data where the number of the majority class is much higher than the number of the minority class:
 - a useless classifier that predicts every sample as the majority class will have almost perfect accuracy (i.e., close to 1)
 - this is because in eq. (76) TP (if positive is the majority class) or TN (if negative is the majority class) will be much larger than the other items or, in other words, the weight of the majority class will be much higher than that of the minority class
 - this results in an accuracy close to 1



Good practice

- It is recommended not to use accuracy to measure the performance of a classifier, particularly when the data is imbalanced.

Precision (Class-Specific)



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

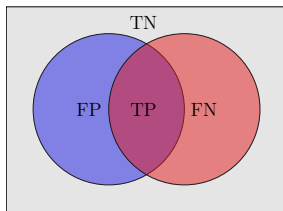
- Based on the TP, FP, TN and FN in fig. 5:
 - Precision of the Positive Class, p_1 , is defined as follows:*

$$p_1 = \frac{TP}{TP + FP} \quad (77)$$

- Precision of the Negative Class, p_0 , is defined as follows:*

$$p_0 = \frac{TN}{TN + FN} \quad (78)$$

Precision (Micro / Macro)



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

- Based on the TP, FP, TN and FN in fig. 5:
 - Precision across the two classes (Micro)*, p_{micro} , is defined as follows:

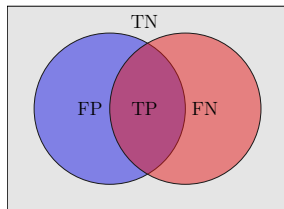
$$p_{\text{micro}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \quad (79)$$

which is the same as accuracy, defined in eq. (76)

- Precision across the two classes (Macro)*, p_{macro} , is defined as follows:

$$p_{\text{macro}} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FP}} + \frac{\text{TN}}{\text{TN} + \text{FN}} \right) \quad (80)$$

Recall (Class-Specific)



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

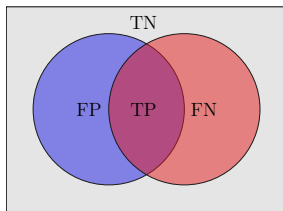
- Based on the TP, FP, TN and FN in fig. 5:
 - Recall of the Positive Class, r_1 , is defined as follows:*

$$r_1 = \frac{TP}{TP + FN} \quad (81)$$

- Recall of the Negative Class, r_0 , is defined as follows:*

$$r_0 = \frac{TN}{TN + FP} \quad (82)$$

Recall (Micro / Macro)



		Predicted Class	
		$\hat{P}$	$\hat{N}$
Actual Class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Figure 5: Venn diagram and confusion matrix showing the TP, FP, TN and FN.

- Based on the TP, FP, TN and FN in fig. 5:
 - Recall across the two classes (Micro)*, r_{micro} , is defined as follows:

$$r_{\text{micro}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \quad (83)$$

which is the same as acc and p_{micro} , defined in eqs. (76) and (79)

- Recall across the two classes (Macro)*, r_{macro} , is defined as follows:

$$r_{\text{macro}} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right) \quad (84)$$

F1-Score (Class-Specific)

- *F1-score of the Positive Class, f_1* , is defined as follows:

$$f_1 = \frac{2p_1r_1}{p_1 + r_1}, \quad (85)$$

where p_1 and r_1 are the precision and recall of the positive class, given in eqs. (77) and (81):

$$p_1 = \frac{TP}{TP + FP}, \quad (77)$$

$$r_1 = \frac{TP}{TP + FN}. \quad (81)$$

- *F1-score of the Negative Class, f_0* , is defined as follows:

$$f_0 = \frac{2p_0r_0}{p_0 + r_0}, \quad (86)$$

where p_0 and r_0 are the precision and recall of the negative class, given in eqs. (78) and (82):

$$p_0 = \frac{TN}{TN + FN}, \quad (80)$$

$$r_0 = \frac{TN}{TN + FP}. \quad (82)$$

F1-Score (Micro / Macro)

- *F1-score across the two Classes (Micro)*, f_{micro} , is defined as follows:

$$f_{\text{micro}} = \frac{2p_{\text{micro}}r_{\text{micro}}}{p_{\text{micro}} + r_{\text{micro}}}, \quad (87)$$

where p_{micro} and r_{micro} are the precision and recall across the two classes (micro), given in eqs. (79) and (83):

$$p_{\text{micro}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \quad (79)$$

$$r_{\text{micro}} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}. \quad (83)$$

- *F1-score across the two Classes (Macro)*, f_{macro} , is defined as follows:

$$f_{\text{macro}} = \frac{2p_{\text{macro}}r_{\text{macro}}}{p_{\text{macro}} + r_{\text{macro}}}, \quad (88)$$

where p_{macro} and r_{macro} are the precision and recall across the two classes (macro), given in eqs. (80) and (84):

$$p_{\text{macro}} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FP}} + \frac{\text{TN}}{\text{TN} + \text{FN}} \right), \quad (80)$$

$$r_{\text{macro}} = \frac{1}{2} \left(\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TN}}{\text{TN} + \text{FP}} \right). \quad (84)$$

Accuracy, Precision, Recall and F1-Score

- It is recommended to report the class-specific precision, recall and F1-score, rather than the accuracy, particularly when the data is imbalanced, since:
 - class-specific metrics allow us to see the classification error in each class
 - accuracy is a summarized metric across all the classes, and gives higher weights to the majority class (leading to a compromised measurement of the classifier)
- If we have to report a summarized score across all the classes, we usually favor macro over micro, particularly when the data is imbalanced, since:
 - micro reduces to accuracy, which gives higher weights to the majority class (leading to a compromised measurement of the classifier)
 - macro gives the same weight to each class, which helps addressing the problem of accuracy and micro



Good practice

- It is recommended to report the class-specific precision, recall and F1-score, rather than the accuracy, particularly when the data is imbalanced.
- If we have to report a summarized score across all the classes, we usually favor macro over micro, particularly when the data is imbalanced.

Precision, Recall and F1-Score: Code

- See [/utilities/p2_shallow_learning:](#)
 - ① cell 10

Proof of KL-divergence Minimum: Page 19

- KL-divergence from $p(y|\theta^*)$ to $p(y|\theta)$, $D_{KL}(p(y|\theta^*)||p(y|\theta))$, is given in eq. (15)

$$D_{KL}(p(y|\theta^*)||p(y|\theta)) = E_{y \sim p(y|\theta^*)} \left[\log \frac{p(y|\theta^*)}{p(y|\theta)} \right]. \quad (15)$$

- When y is discrete, we can write eq. (15) as

$$D_{KL}(p(y|\theta^*)||p(y|\theta)) = \sum_y p(y|\theta^*) \log \frac{p(y|\theta^*)}{p(y|\theta)}. \quad (89)$$

- Since for each $x \geq 0$ we have

$$-\log x \geq 1 - x, \quad (90)$$

where the equality holds when $x = 1$, we can write eq. (89) as

$$\begin{aligned} D_{KL}(p(y|\theta^*)||p(y|\theta)) &= \sum_y p(y|\theta^*) \log \frac{p(y|\theta^*)}{p(y|\theta)}, \\ &= - \sum_y p(y|\theta^*) \log \frac{p(y|\theta)}{p(y|\theta^*)}, \\ &\geq - \sum_y p(y|\theta^*) \left(1 - \frac{p(y|\theta)}{p(y|\theta^*)} \right) = - \sum_y p(y|\theta^*) - \sum_y p(y|\theta) = 0, \end{aligned} \quad (91)$$

where the equality holds (i.e., KL-divergence takes the minimum value, 0) when $p(y|\theta^*) = p(y|\theta)$, which proves the claim in eq. (16) on page 19.

Proof of Minimizing KL-divergence Equates MLE: Page 19

- KL-divergence from $p(y|\theta^*)$ to $p(y|\theta)$, $D_{KL}(p(y|\theta^*)||p(y|\theta))$, is given in eq. (15)

$$D_{KL}(p(y|\theta^*)||p(y|\theta)) = E_{y \sim p(y|\theta^*)} \left[\log \frac{p(y|\theta^*)}{p(y|\theta)} \right]. \quad (15)$$

- We can write eq. (15) as

$$\begin{aligned} D_{KL}(p(y|\theta^*)||p(y|\theta)) &= E_{y \sim p(y|\theta^*)} \left[\log \frac{p(y|\theta^*)}{p(y|\theta)} \right], \\ &= E_{y \sim p(y|\theta^*)} [\log p(y|\theta^*)] - E_{y \sim p(y|\theta^*)} [\log p(y|\theta)]. \end{aligned} \quad (92)$$

- Since the second item in eq. (92), $E_{y \sim p(y|\theta^*)} [\log p(y|\theta)]$, is constant, minimizing the KL-divergence (first item in eq. (92)), $D_{KL}(p(y|\theta^*)||p(y|\theta))$, equates maximizing the third item in the equation, $E_{y \sim p(y|\theta^*)} [\log p(y|\theta)]$ and, in turn, maximizing the likelihood, $p(y|\theta)$:

$$\arg \min_{\theta} D_{KL}(p(y|\theta^*)||p(y|\theta)) = \arg \max_{\theta} E_{y \sim p(y|\theta^*)} [\log p(y|\theta)] = \arg \max_{\theta} p(y|\theta), \quad (93)$$

which proves the claim on page 19.

Proof of Binomial Cross Entropy: Page 20

- As discussed earlier, we can use MLE (eq. (14)) to estimate the parameters, $\boldsymbol{\theta} = [b \quad w_1 \cdots w_n]^\top$:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \binom{m}{k} \prod_{i=1}^m (p^i)^{y^i} (1 - p^i)^{1-y^i}. \quad (14)$$

- Since $\binom{m}{k}$ is a constant with respect to $\boldsymbol{\theta}$, we can write eq. (14) as

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^m (p^i)^{y^i} (1 - p^i)^{1-y^i}. \quad (94)$$

- Since maximizing $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$ equates maximizing $\log \mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$, we can write eq. (94) as

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \log \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \log \left(\prod_{i=1}^m (p^i)^{y^i} (1 - p^i)^{1-y^i} \right). \quad (95)$$

Proof of Binomial Cross Entropy: Page 20

- Based on the following law of logarithms:

$$\log ab = \log a + \log b, \quad (96)$$

we can write eq. (95) as

$$\begin{aligned} \boldsymbol{\theta}^* &= \arg \max_{\boldsymbol{\theta}} \log \left(\prod_{i=1}^m (p^i)^{y^i} (1 - p^i)^{1-y^i} \right), \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \log \left((p^i)^{y^i} (1 - p^i)^{1-y^i} \right), \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \left(\log (p^i)^{y^i} + \log (1 - p^i)^{1-y^i} \right). \end{aligned} \quad (97)$$

Proof of Binomial Cross Entropy: Page 20

- Based on the following law of logarithms:

$$\log a^b = b \log a, \quad (98)$$

we can write eq. (97) as

$$\begin{aligned} \theta^* &= \arg \max_{\theta} \sum_{i=1}^m \left(\log (p^i)^{y^i} + \log (1 - p^i)^{1-y^i} \right), \\ &= \arg \max_{\theta} \sum_{i=1}^m \left(y^i \log (p^i) + (1 - y^i) \log (1 - p^i) \right), \\ &= \arg \min_{\theta} \left(- \sum_{i=1}^m \left(y^i \log (p^i) + (1 - y^i) \log (1 - p^i) \right) \right), \end{aligned} \quad (99)$$

which proves the claim in eq. (17) on page 20.

Proof of the Updating Rule: Page 25

- We can write the gradient (i.e., first-order derivative) of the loss function with respect to mini-batch $\mathbf{mb}^j$, $\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top$, as

$$\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top = \left[\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j) \quad \frac{\partial}{\partial w_1} \mathcal{L}(\boldsymbol{\theta}^j) \cdots \frac{\partial}{\partial w_n} \mathcal{L}(\boldsymbol{\theta}^j) \right]^\top, \quad (100)$$

where the loss function, $\mathcal{L}(\boldsymbol{\theta}^j)$, is given in eq. (22)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)). \quad (22)$$

Proof of the Updating Rule: Page 25

- Based on eq. (22), we can write $\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j)$ as

$$\begin{aligned} \frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j) &= -\frac{\partial}{\partial b} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)) \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y^i \frac{\partial \log(p^i)}{\partial b} + (1 - y^i) \frac{\partial \log(1 - p^i)}{\partial b} \right). \end{aligned} \quad (101)$$

- We now derive the two partial derivatives in eq. (101), $\frac{\partial \log(p^i)}{\partial b}$ and $\frac{\partial \log(1 - p^i)}{\partial b}$.
- Based on the derivative rule of Logarithms, we can write the two partial derivatives as

$$\frac{\partial \log(p^i)}{\partial b} = \frac{1}{p^i} \frac{\partial p^i}{\partial b}, \quad (102)$$

$$\frac{\partial \log(1 - p^i)}{\partial b} = \frac{1}{1 - p^i} \frac{\partial}{\partial b} (1 - p^i) = -\frac{1}{1 - p^i} \frac{\partial p^i}{\partial b}. \quad (103)$$

- We now derive the common item in eqs. (102) and (103), $\frac{\partial p^i}{\partial b}$.

Proof of the Updating Rule: Page 25

- Based on eq. (25)

$$p^i = \frac{1}{1 + e^{-n^i}} \quad \text{where} \quad n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta}^J, \quad (25)$$

we can write $\frac{\partial p^i}{\partial b}$ in eqs. (102) and (103) as

$$\begin{aligned} \frac{\partial (p^i)}{\partial b} &= \frac{\partial \frac{1}{1+e^{-n^i}}}{\partial b}, \\ &= -\frac{1}{(1+e^{-n^i})^2} \frac{\partial}{\partial b} (1+e^{-n^i}), \\ &= \frac{e^{-n^i}}{(1+e^{-n^i})^2} \frac{\partial n^i}{\partial b}. \end{aligned} \quad (104)$$

Proof of the Updating Rule: Page 25

- Based on eq. (25), we can write eq. (104) as

$$\begin{aligned}
 \frac{\partial (p^i)}{\partial b} &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2} \frac{\partial n^i}{\partial b}, \\
 &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2} \frac{\partial}{\partial b} (b + w_1 x_1^i + \cdots + w_n x_n^i), \\
 &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2}, \\
 &= \left(\frac{1 + e^{-n^i}}{(1 + e^{-n^i})^2} - \frac{1}{(1 + e^{-n^i})^2} \right), \\
 &= (p^i - (p^i)^2), \\
 &= p^i (1 - p^i).
 \end{aligned} \tag{105}$$

Proof of the Updating Rule: Page 25

- By substituting eq. (105) into eqs. (102) and (103), we have

$$\frac{\partial \log(p^i)}{\partial b} = \frac{1}{p^i} \frac{\partial p^i}{\partial b} = \frac{1}{p^i} p^i (1 - p^i) = 1 - p^i, \quad (106)$$

$$\frac{\partial \log(1 - p^i)}{\partial b} = -\frac{1}{1 - p^i} \frac{\partial p^i}{\partial b} = -\frac{1}{1 - p^i} (1 - p^i) p^i = -p^i. \quad (107)$$

- By substituting eqs. (106) and (107) into eq. (101), we have

$$\begin{aligned} \frac{\partial}{\partial b} \mathcal{L}(\theta^j) &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y^i \frac{\partial \log(p^i)}{\partial b} + (1 - y^i) \frac{\partial \log(1 - p^i)}{\partial b} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i (1 - p^i) + (1 - y^i) (-p^i)), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - y^i p^i - p^i + y^i p^i), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i). \end{aligned} \quad (108)$$

Proof of the Updating Rule: Page 25

- Based on eq. (22), we can write $\frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^j)$ as

$$\begin{aligned} \frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^j) &= -\frac{\partial}{\partial w_j} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i \log(p^i) + (1 - y^i) \log(1 - p^i)) \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y^i \frac{\partial \log(p^i)}{\partial w_j} + (1 - y^i) \frac{\partial \log(1 - p^i)}{\partial w_j} \right). \end{aligned} \quad (109)$$

- We now derive the two partial derivatives in eq. (109), $\frac{\partial \log(p^i)}{\partial w_j}$ and $\frac{\partial \log(1 - p^i)}{\partial w_j}$.
- Based on the derivative rule of Logarithms, the two partial derivatives can be written as

$$\frac{\partial \log(p^i)}{\partial w_j} = \frac{1}{p^i} \frac{\partial p^i}{\partial w_j}, \quad (110)$$

$$\frac{\partial \log(1 - p^i)}{\partial w_j} = \frac{1}{1 - p^i} \frac{\partial}{\partial w_j} (1 - p^i) = -\frac{1}{1 - p^i} \frac{\partial p^i}{\partial w_j}. \quad (111)$$

- We now derive the common item in eqs. (110) and (111), $\frac{\partial p^i}{\partial w_j}$.

Proof of the Updating Rule: Page 25

- Based on eq. (25)

$$p^i = \frac{1}{1 + e^{-n^i}} \quad \text{where} \quad n^i = b + w_1 x_1^i + \cdots + w_n x_n^i = [1 \quad \mathbf{x}^i] \boldsymbol{\theta}^j, \quad (25)$$

we can write $\frac{\partial p^i}{\partial w_j}$ in eqs. (110) and (111) as

$$\begin{aligned} \frac{\partial (p^i)}{\partial w_j} &= \frac{\partial \frac{1}{1+e^{-n^i}}}{\partial w_j}, \\ &= -\frac{1}{(1 + e^{-n^i})^2} \frac{\partial}{\partial w_j} (1 + e^{-n^i}), \\ &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2} \frac{\partial n^i}{\partial w_j}. \end{aligned} \quad (112)$$

Proof of the Updating Rule: Page 25

- Based on eq. (25), we can write eq. (112) as

$$\begin{aligned}
 \frac{\partial (p^i)}{\partial w_j} &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2} \frac{\partial n^i}{\partial w_j}, \\
 &= \frac{e^{-n^i}}{(1 + e^{-n^i})^2} \frac{\partial}{\partial w_j} (b + w_1 x_1^i + \cdots + w_n x_n^i), \\
 &= \left(\frac{e^{-n^i}}{(1 + e^{-n^i})^2} \right) x_j^i, \\
 &= \left(\frac{1 + e^{-n^i}}{(1 + e^{-n^i})^2} - \frac{1}{(1 + e^{-n^i})^2} \right) x_j^i, \\
 &= (p^i - (p^i)^2) x_j^i, \\
 &= p^i (1 - p^i) x_j^i.
 \end{aligned} \tag{113}$$

Proof of the Updating Rule: Page 25

- By substituting eq. (113) into eqs. (110) and (111), we have

$$\frac{\partial \log(p^i)}{\partial w_j} = \frac{1}{p^i} \frac{\partial p^i}{\partial w_j} = \frac{1}{p^i} p^i (1 - p^i) x_j^i = (1 - p^i) x_j^i, \quad (114)$$

$$\frac{\partial \log(1 - p^i)}{\partial w_j} = -\frac{1}{1 - p^i} \frac{\partial p^i}{\partial w_j} = -\frac{1}{1 - p^i} (1 - p^i) p^i x_j^i = -p^i x_j^i. \quad (115)$$

- By substituting eqs. (114) and (115) into eq. (109), we have

$$\begin{aligned} \frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^j) &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y^i \frac{\partial \log(p^i)}{\partial w_j} + (1 - y^i) \frac{\partial \log(1 - p^i)}{\partial w_j} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i (1 - p^i) + (1 - y^i) (-p^i)) x_j^i, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - y^i p^i - p^i + y^i p^i) x_j^i, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i) x_j^i. \end{aligned} \quad (116)$$

Proof of the Updating Rule: Page 25

- Based on eqs. (108) and (116), we can write eq. (100) as

$$\begin{aligned}
 \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top &= \left[\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j) \quad \frac{\partial}{\partial w_1} \mathcal{L}(\boldsymbol{\theta}^j) \cdots \frac{\partial}{\partial w_n} \mathcal{L}(\boldsymbol{\theta}^j) \right]^\top, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i) \begin{bmatrix} 1 & x_1^i & \cdots & x_n^i \end{bmatrix}^\top, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top.
 \end{aligned} \tag{117}$$

- By substituting eq. (117) into eq. (23), we have

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - p^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix}^\top (\mathbf{y}^j - \mathbf{p}^j), \tag{118}$$

which proves the claim in eq. (24) on page 25. □

Proof of Multinomial Cross Entropy: Page 46

- As discussed earlier, we can use MLE (eq. (52)) to estimate the parameters, $\boldsymbol{\theta} = [\mathbf{b} \quad \mathbf{w}_1 \cdots \mathbf{w}_n]^\top$:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \binom{n}{n_1, \dots, n_c} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}. \quad (52)$$

- Since $\binom{n}{n_1, \dots, n_c}$ is a constant with respect to $\boldsymbol{\theta}$, we can write eq. (52) as

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i}. \quad (119)$$

- Since maximizing $\mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$ equates maximizing $\log \mathbf{p}(\mathbf{y}|\boldsymbol{\theta})$, we can write eq. (119) as

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \log \mathbf{p}(\mathbf{y}|\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \log \left(\prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i} \right). \quad (120)$$

Proof of Multinomial Cross Entropy: Page 46

- Based on the law of logarithms in eq. (96):

$$\log ab = \log a + \log b, \quad (96)$$

we can write eq. (120) as

$$\begin{aligned} \boldsymbol{\theta}^* &= \arg \max_{\boldsymbol{\theta}} \log \left(\prod_{i=1}^m \prod_{k=1}^c (p_k^i)^{y_k^i} \right), \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \sum_{k=1}^c \log \left((p_k^i)^{y_k^i} \right). \end{aligned} \quad (121)$$

Proof of Multinomial Cross Entropy: Page 46

- Based on the law of logarithms in eq. (98):

$$\log a^b = b \log a, \quad (98)$$

we can write eq. (121) as

$$\begin{aligned} \boldsymbol{\theta}^* &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \sum_{k=1}^c \log \left((p_k^i)^{y_k^i} \right), \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \sum_{k=1}^c y_k^i \log (p_k^i), \\ &= \arg \min_{\boldsymbol{\theta}} \left(- \sum_{i=1}^m \sum_{k=1}^c y_k^i \log (p_k^i) \right), \end{aligned} \quad (122)$$

which proves the claim in eq. (53) on page 46.

Proof of the Updating Rule: Page 51

- The gradient (i.e., first-order derivative) of the loss function with respect to mini-batch $\mathbf{mb}^j$ and class k , $\nabla \mathcal{L}^k(\boldsymbol{\theta}^j)^\top$, can be written as

$$\nabla \mathcal{L}^k(\boldsymbol{\theta}^j)^\top = \left[\frac{\partial}{\partial b^k} \mathcal{L}(\boldsymbol{\theta}^j) \quad \frac{\partial}{\partial w_1^k} \mathcal{L}(\boldsymbol{\theta}^j) \cdots \frac{\partial}{\partial w_n^k} \mathcal{L}(\boldsymbol{\theta}^j) \right]^\top, \quad (123)$$

where the loss function, $\mathcal{L}(\boldsymbol{\theta}^j)$, is given in eq. (56)

$$\mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{l=1}^c y_l^i \log(p_l^i). \quad (56)$$

Proof of the Updating Rule: Page 51

- Based on eq. (56), we can write $\frac{\partial}{\partial b^k} \mathcal{L}(\boldsymbol{\theta}^j)$ as

$$\begin{aligned}
 \frac{\partial}{\partial b^k} \mathcal{L}(\boldsymbol{\theta}^j) &= -\frac{\partial}{\partial b^k} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{l=1}^c y_l^i \log(p_l^i) \right), \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{l=1}^c y_l^i \frac{\partial \log(p_l^i)}{\partial b^k}, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} y_l^i \frac{\partial \log(p_l^i)}{\partial b^k} + y_k^i \frac{\partial \log(p_k^i)}{\partial b^k} \right).
 \end{aligned} \tag{124}$$

- Based on the derivative rule of Logarithms, we can write eq. (124) as

$$\frac{\partial}{\partial b^k} \mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial (p_l^i)}{\partial b^k} + \frac{y_k^i}{p_k^i} \frac{\partial (p_k^i)}{\partial b^k} \right). \tag{125}$$

Proof of the Updating Rule: Page 51

Based on eqs. (50) and (51)

$$p_q^i = \frac{e^{n_q^i}}{\sum_{m=1}^c e^{n_m^i}}, \quad \text{where} \quad n_q^i = b^q + w_1^q x_1^i + \cdots + w_n^q x_n^i = [1 \quad x^i] \theta_q^j, \quad (50,51)$$

we can write eq. (125) as

$$\begin{aligned} \frac{\partial}{\partial b^k} \mathcal{L}(\theta^j) &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial(p_l^i)}{\partial b^k} + \frac{y_k^i}{p_k^i} \frac{\partial(p_k^i)}{\partial b^k} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial}{\partial b^k} \frac{e^{n_l^i}}{\sum_{m=1}^c e^{n_m^i}} + \frac{y_k^i}{p_k^i} \frac{\partial}{\partial b^k} \frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{e^{n_l^i} e^{n_k^i}}{\left(\sum_{m=1}^c e^{n_m^i}\right)^2} + \frac{y_k^i}{p_k^i} \left(\frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} - \left(\frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} \right)^2 \right) \right) \frac{\partial n_k^i}{\partial b^k}, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} \frac{y_l^i}{p_l^i} p_l^i p_k^i + \frac{y_k^i}{p_k^i} \left(p_k^i - (p_k^i)^2 \right) \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} y_l^i p_k^i + y_k^i (1 - p_k^i) \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y_k^i - p_k^i \left(\sum_{l \neq k} y_l^i + y_k^i \right) \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y_k^i - p_k^i). \end{aligned} \quad (126)$$

Proof of the Updating Rule: Page 51

- Based on eq. (56), we can write $\frac{\partial}{\partial w_j^k} \mathcal{L}(\boldsymbol{\theta}^j)$ as

$$\begin{aligned}
 \frac{\partial}{\partial w_j^k} \mathcal{L}(\boldsymbol{\theta}^j) &= -\frac{\partial}{\partial w_j^k} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{l=1}^c y_l^i \log(p_l^i) \right), \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \sum_{l=1}^c y_l^i \frac{\partial \log(p_l^i)}{\partial w_j^k}, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} y_l^i \frac{\partial \log(p_l^i)}{\partial w_j^k} + y_k^i \frac{\partial \log(p_k^i)}{\partial w_j^k} \right).
 \end{aligned} \tag{127}$$

- Based on the derivative rule of Logarithms, we can write eq. (127) as

$$\frac{\partial}{\partial w_j^k} \mathcal{L}(\boldsymbol{\theta}^j) = -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial (p_l^i)}{\partial w_j^k} + \frac{y_k^i}{p_k^i} \frac{\partial (p_k^i)}{\partial w_j^k} \right). \tag{128}$$

Proof of the Updating Rule: Page 51

Based on eqs. (50) and (51)

$$p_q^i = \frac{e^{n_q^i}}{\sum_{m=1}^c e^{n_m^i}}, \quad \text{where} \quad n_q^i = b^q + w_1^q x_1^i + \cdots + w_n^q x_n^i = [1 \quad x^i] \theta_q^j, \quad (50,51)$$

we can write eq. (128) as

$$\begin{aligned} \frac{\partial}{\partial w_j^k} \mathcal{L}(\theta^j) &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial(p_l^i)}{\partial w_j^k} + \frac{y_k^i}{p_k^i} \frac{\partial(p_k^i)}{\partial w_j^k} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{\partial}{\partial w_j^k} \frac{e^{n_l^i}}{\sum_{m=1}^c e^{n_m^i}} + \frac{y_k^i}{p_k^i} \frac{\partial}{\partial w_j^k} \frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} \right), \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} \frac{y_l^i}{p_l^i} \frac{e^{n_l^i} e^{n_k^i}}{\left(\sum_{m=1}^c e^{n_m^i}\right)^2} + \frac{y_k^i}{p_k^i} \left(\frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} - \left(\frac{e^{n_k^i}}{\sum_{m=1}^c e^{n_m^i}} \right)^2 \right) \right) \frac{\partial n_k^i}{\partial w_j^k}, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} \frac{y_l^i}{p_l^i} p_l^i p_k^i + \frac{y_k^i}{p_k^i} \left(p_k^i - (p_k^i)^2 \right) \right) x_j^i, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(-\sum_{l \neq k} y_l^i p_k^i + y_k^i (1 - p_k^i) \right) x_j^i, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y_k^i - p_k^i \left(\sum_{l \neq k} y_l^i + y_k^i \right) \right) x_j^i, \\ &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} \left(y_k^i - p_k^i \right) x_j^i. \end{aligned} \quad (129)$$

Proof of the Updating Rule: Page 51

- Based on eqs. (126) and (129), we can write eq. (123) as

$$\begin{aligned}
 \nabla \mathcal{L}^k(\boldsymbol{\theta}^j)^\top &= \left[\frac{\partial}{\partial b^k} \mathcal{L}(\boldsymbol{\theta}^j) \quad \frac{\partial}{\partial w_1^k} \mathcal{L}(\boldsymbol{\theta}^j) \cdots \frac{\partial}{\partial w_n^k} \mathcal{L}(\boldsymbol{\theta}^j) \right]^\top, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y_k^i - p_k^i) [1 \quad x_1^i \cdots x_n^i]^\top, \\
 &= -\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y_k^i - p_k^i) [1 \quad \mathbf{x}^i]^\top.
 \end{aligned} \tag{130}$$

- By substituting eq. (130) into eq. (23), we have

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{\eta_k}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{Y}^j - \mathbf{P}^j), \tag{131}$$

which proves the claim in eq. (57) on page 51. □

Bibliography I



Masters, D. and Luschi, C. (2018).

Revisiting Small Batch Training for Deep Neural Networks.

arXiv preprint arXiv:1804.07612.